
practice, intervention hyperparameters are stable. We include examples of failed steering vectors in Appendix Table 7. We also have not examined ActAdd’s potential impact on reasoning. ActAdd is not immediately applicable given only API access to a model. The model must both cache and expose intermediate activations at the given layer (Bloom & Nanda 2022). Currently, APIs generally do not allow for this.

Activation engineering vs finetuning Finetuning is better understood and more flexible – we doubt that activation engineering can e.g. teach a model a new skill. However, finetuning is significantly more costly and may not be able to elicit the same kinds of capabilities which activation engineering can elicit.

The first advantage of ActAdd is efficiency: the method requires no backward passes and can thus run on any machine that can perform inference rather than training. Implementation effort is also greatly reduced; only forward passes are required to find a suitable (p_+, p_-) and minimal labelled data is required - just the steering prompt pair. We discovered most of the example contrast pairs in Appendix Table 6 in minutes. All things considered, even nontechnical users can benefit from rapid feedback and relatively easy iteration

Activation engineering vs prompt engineering Activation additions can be continuously weighted, while prompts are discrete – a token is either present, or not. To more intensely steer the model to generate wedding-related text, our method does not require any edit to the prompt, but instead just increasing the injection coefficient. See Appendix B for suggestive experiments on ActAdd vs prompting. Unlike system prompts, activation additions do not take up token space in the model’s context window, although this is a small benefit in the era of multi-million token context windows.

While prompting is more flexible and even cheaper than ActAdd, activation additions may elicit capabilities which prompting cannot (as evidenced by our superior results over prompting; see also the speculation in Section 1).

Interpretability of LLMs In most programs, adding values to imprecisely targeted intermediate memory locations would not yield sensible results. Why expect this from Transformers?

A growing consensus is that the activation space of an LLM contains directions which represent high-level latents causally involved in what is generated (Burns et al. 2022; Moschella et al. 2023; Li et al. 2023a; Nanda 2023; Li et al. 2023b).

Our hypothesis, following Elhage et al. 2022, is more specific: that neural networks represent features of the input as directions in activation space, that is, with a linear representation (Park et al. 2023). Moreover, the direction in activation space that corresponds to (say) a love-hate latent variable stays approximately the *same* across a broad class of inputs.

Alain & Bengio 2018 use linear probes on residual streams to infer that LLM representations are at least partially linear; if a linear probe can predict some feature of text output from the residuals with high accuracy, this forms evidence that the feature is represented linearly (i.e. as a simple direction) (Nanda 2023).

The success of activation addition gives stronger, experimental evidence of feature linearity, demonstrating that models *use* feature-related information. Consider the central Love – Hate vector example: we add it to the forward pass and so increase love-related completions. On the examined prompts, this direction is responsible for steering the rest of the model towards love-related completions. In general, steering vectors establish *causality*, at least in the limited set of contexts examined.

Value alignment of LLMs Activation engineering is a promising way to control LLMs. Successor methods may be able to provide general steering methods (e.g. through some analogue of a `Be helpful` vector). Alongside contemporaneous work (Li et al. 2023b; Liu et al. 2023), our experiments suggest that activation engineering can flexibly retarget LLM behavior without damaging general performance. We speculate that ActAdd changes the model’s currently active mixture of goals and priorities. Suitably developed, the activation engineering approach could enable safety progress while preserving overall capabilities

6 CONCLUSION

While methods like prompt engineering, controlled decoding, and finetuning have benefits, they fail to elicit full capabilities from language models. To more reliably elicit these abilities, *activation engineering* strategically perturbs activations at inference time. In particular, we introduced *Activation Addition* to steer models by shifting their inference-time activations along a certain direction (like the “Love”-“Hate” vector). ActAdd is lightweight and effective, achieving SOTA on toxicity reduction and sentiment shift while retaining overall model capabilities. ActAdd demonstrates the promise of activation engineering. We look forward to future work realizing this promise.

REPRODUCIBILITY STATEMENT

Our code is available here: <https://zenodo.org/records/13879423>. The following is an exhaustive list of models used, sampling strategies used, and searches run:

Data processing To curate a wedding-related subset of OpenWebText, we retained documents with wedding-related words (see Section 4.1.1). The only pre-processing performed is to remove sequences of null characters. Each document is split into sentences $s_j \in d_i$ using the Punkt tokenizer (Strunk 2013).

Models After observing success with GPT-2-XL, to replicate our results, we subsequently repeated the same experiments with Llama-1-13B (Touvron et al. 2023) and GPT-J-6B (Wang & Komatsuzaki 2021). Our toxicity and sentiment experiments use OPT (Zhang et al. 2022b) and LLaMA-3-8B Meta 2024. See Appendix E for details. We use `all-MiniLM-L6-v2` (Reimers & Gurevych 2019) to compute sentence embeddings to calculate relevance using cosine similarity. For the success score, we use the SiBERT (Hartmann et al. 2023) sentiment classifier. We perform sentiment classification with the SiBERT classifier (Hartmann et al., 2023).

APIs For scoring toxicity, we use <https://www.perspectiveapi.com/>. For scoring fluency, we use OpenAI `davinci-002`. The PREADD baseline instead used the discontinued `davinci-001` model.

Seed We ran all generations on seed 0. After collecting all other data, we validated that our qualitative results transfer to seeds 1 and 2.

Sampling hyperparameters We precommitted to fixed sampling hyperparameters, selected before experiments began. We held them fixed throughout our data collection. Those sampling hyperparameters were `temperature= 1.0`, `freq_penalty= 1.0`, and `top_p=0.3`. Since this `top_p` value seemed a bit unusual to us in retrospect, we invited an external researcher to reproduce this process with an *unmodified* GPT-2-XL and report the best sampling hyperparameters they found. This second experiment was blinded, as they did not know the values we used. They found that `temperature= 0.6` and `top_p= 0.5` produced better GPT-2-XL capabilities. We reran all our qualitative results at this setting, and they all reproduced (subjectively, more impressively).

We use the same sampling hyperparameters for the toxicity and sentiment experiments. Numbers reported by the other authors were obtained with `freq_penalty= 0.0`, and `top_p=1.0`.

In replicating the unsteered OPT sentiment baseline, we find that the NegToPos direction is consistently higher success than PosToNeg. This holds across different combinations of model hyperparameters, including those in Pei et al. 2023. However, PREADD Pei et al., 2023 reports similar success results for both (i.e. a much lower NegToPos success). The OPT results use our calculated values.

Reporting the best of K completions We generated $K = 3$ completions for each qualitative demonstration, for both normal and steered forward-passes. Appendix Table 6, shows the subjectively most compelling completion pair out of the *first* three seed-0 completion-pairs. You can see all top-3 completions for the entries in this notebook: tinyurl.com/actadd3. We share activation additions which work well. We iterated over contrast pairs to get these to work, although several striking results were generated within [first author’s] first hour of using the technique. Out of the 12 activation additions we thought demonstrated a distinct ability of the method, we decided not to include 1 because its first three seed-0 completions were unusually unimpressive. We include the remaining 11 in Table 6.

ActAdd hyperparameters (l, c) *This section does not have complete statistics.* We perform simple grid search, usually between $c \in [3, 20]$ and $l \in [6, 24]$.

Hardware: GPU: Nvidia RTX A5000, CPU: Intel Core i9-10900X CPU @ 3.70GHz. 24GB GPU RAM, 32GB system RAM

Relevant libraries and frameworks: Operating system: Ubuntu 22.04.1 LTS, numpy: 1.24.3, pandas: 2.0.1, torch: 1.13.1, transformer-lens: 1.4.0.

AUTHOR CONTRIBUTIONS

Turner: conceptualization, team management, implementation of core features, design of many experiments, discovery of many individual steering vectors, and wrote much of the original post.

Thiergart: had idea for variations on positions of addition, implemented the positional experiment, worked on theory.

Leech: designed new experiments, designed figures, formalized the algorithm and evaluations, wrote the main text based on the earlier post, literature review.

MacDiarmid: most of the main library code.

Udell: wrote and edited the original post, generated qualitative results.

Mini: infrastructure support, OpenAI wrappers, experiments on LLaMA, Vicuna and GPT-J.

Vazquez: wrote part of text, conducted toxicity, sentiment control, and other experiments on LLaMA-3, OPT, GPT-2.

ACKNOWLEDGMENTS

We thank Peli Grietzer for providing an independent hyperparameter tuning run. We thank Alex Cloud, Jan Brauner, Andis Draguns, Sören Mindermann and Raymond Douglas for helpful comments on the draft, as well as Andrew Critch, Aryan Bhatt, Chris Olah, Ian McKenzie, janus, Julian Schulz, Justis Mills, Lawrence Chan, Leo Gao, Neel Nanda, Oliver Habryka, Olivia Jimenez, Paul Christiano, Peter Barnett, Quintin Pope, Tamera Lanham, Thomas Kwa, and Tristan Hume for comments on an earlier draft. We thank Rusheb Shah for engineering assistance. We thank Garrett Baker for running tests on GPT-J (6B) We thank an anonymous ICML reviewer for their extremely thoughtful comments.

REFERENCES

- Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2018.
- Joseph Bloom and Neel Nanda. TransformerLens: A library for mechanistic interpretability of generative language models. <https://neelnanda-io.github.io/TransformerLens/>, 2022.
- Davis Brown, Charles Godfrey, Cody Nizinski, Jonathan Tu, and Henry Kvinge. Robustness of edited neural networks, 2023.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision, 2022.
- Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. Plug and play language models: A simple approach to controlled text generation, 2020.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 1, 2021.